CIVICPULSE

A Lightweight Java System for Observing and Improving Public Service Operations



MAIN PROBLEM STATEMENT

To observe public service usage and demonstrate a very basic system for efficient operation.

SUB PROBLEM STATEMENT

Designing a lightweight, privacy-preserving Java backend that ingests raw telemetry from public-infrastructure sensors (bridges, utility lines, streetlights), anonymizes it in memory, detects operational anomalies through diffing against historical baselines, and surfaces the results as plain-language maintenance alerts — without requiring any custom firmware or on-device intelligence.

Prabhav Dwiwedi

Developed for Idea Lab | Semester 5 | Third Year

OVERVIEW

System Architecture

Edge devices stay dumb. All intelligence — anonymization, diffing, storage, and language generation — runs inside a single Java backend application.

Edge Sensors

Stock LoRaWAN / MQTT telemetry units. No custom firmware, no on-device logic — plain scalar readings only.

LoRaWAN / HTTPS
MQTT

CivicPulse Java Backend

Packaged as a Spring Boot / Quarkus / Javalin service · runs on any standard JVM host

1. **In-Memory Anonymizer**

   Strips PII, MAC addresses and fine-grained metadata on arrival; keeps only scalar metrics (vibration RMS, temperature, RF power).

2. **Java Diff Engine (java-diff-utils)**

   Computes deltas between the incoming scalar window and the stored baseline using a Myers-style shortest-edit-script algorithm.

3. **Yggdrasil Storage**

   Embedded, zero-dependency Java log store (B+Tree engine) persisting numeric deltas with write-ahead logging and authenticated pages.

4. **Gemini API Service**

   Converts persisted delta logs into structured, plain-language maintenance alerts for city operations staff.

SHREE L.R. TIWARI College of Engineering

DESIGN PRINCIPLE

A Fully Java-Native Pipeline

No embedded C++/Rust firmware, no on-device JVM, no micro-runtime mismatch. Every stage — ingest to insight — runs on one standard JVM.

Network Decoupling

Edge hardware ships stock firmware and reports raw readings over LoRaWAN/HTTPS. No custom firmware or no_std Rust to build or maintain.

Privacy-First Ingest

Raw payloads are sanitized the moment they hit Java heap memory — PII, device IDs and fine-grained metadata are dropped before anything is written to disk.

Deterministic Diffing

java-diff-utils (Myers shortest-edit-script algorithm) computes deltas between the current sensor window and its historical baseline in plain Java, no native bindings.

Portable Deployment

The service packages as a standard Spring Boot / Quarkus / Javalin JAR and runs on any cloud provider or on-premise server — no specialised edge compute.

STORAGE LAYER — ALREADY BUILT

Yggdrasil-Blind_Storage

A zero-dependency, embeddable Java storage engine used as CivicPulse's local log store for numeric deltas. Built and tested independently of this project.

Module Layout

```
Yggdrasil-Blind_Storage/
 README.md · BUG_FIX_REPORT.md
 web/index.html   (dashboard UI)

 src/main/java/Yggdrasil/
  NioServer · DashboardServer
  RequestDispatcher · WireProtocol
  ConnectionContext · FrameAccumulator

  core/
   BPlusTree · InternalNode · LeafNode
   HeapFile · SlottedPage · Page
   RecordManager · StorageManager
   WalManager · Metadata · TupleID

  crypto/
   Ascon128Engine

  gui/
   SupervisorConsole · ControlPanel
   BPlusTreeCanvas · SecurityPanel
   HexInspectorPanel · EngineTelemetry
```

Zero-Dependency Core

The storage engine (B+Tree index, heap files, slotted pages, WAL) is written in plain Java with no external libraries — it drops straight into the CivicPulse JVM.

Durability & Integrity

A write-ahead log (WalManager) and page-level authentication (PageAuthenticationException) protect delta records against partial writes and tampering.

Ascon-128 Encryption

Lightweight authenticated encryption (NIST-selected Ascon cipher) secures stored delta pages at rest, matching the privacy-first design of the ingest layer.

Live Operator Console

A NIO server plus a Swing-based Supervisor Console and web dashboard expose real-time B+Tree and engine telemetry for inspection during testing.

DEPENDENCY INTEGRATION

Diffing with java-diff-utils

Anomaly detection reduces to comparing an incoming scalar sensor window against its stored baseline. java-diff-utils is added as a plain Maven/Gradle dependency — no native code.

Maven Dependency

```
<dependency>
  <groupId>io.github.java-diff-utils</groupId>
  <artifactId>java-diff-utils</artifactId>
  <version>4.15</version>
</dependency>
```

Baseline vs. Live Window

```
Patch<String> patch =
  DiffUtils.diff(baselineWindow, liveWindow);

patch.getDeltas().forEach(delta ->
  yggdrasil.append(delta.toString())
);
```

How the Engine Works

1 Myers shortest-edit-script core

java-diff-utils implements the classic O(ND) algorithm (Myers, 1986) to find the minimal set of insertions/deletions between two sequences.

2 Applied to scalar windows

CivicPulse treats each sensor's rolling window of scalar readings as a sequence and diffs it against the stored baseline sequence.

3 Deltas, not raw streams

Only the computed Delta objects — not full readings — are persisted to Yggdrasil, keeping storage lean and privacy-preserving.

4 Threshold-triggered alerts

When delta magnitude crosses a configured threshold, the Gemini API Service is invoked to draft a plain-language maintenance note.

System Demonstration Flow

A working end-to-end path shows the full pipeline operating on synthetic telemetry before any field deployment.

1

Synthetic Telemetry Generator

A scheduled Java service simulates multiple urban infrastructure nodes, including deliberate anomaly injection (e.g. a vibration spike or sudden temperature drop).

2

Live Anonymization & Diffing

Each simulated reading is stripped of identifying fields, then run through java-diff-utils to surface drift from the stored baseline.

3

Yggdrasil Delta Persistence

Computed deltas are written to the Yggdrasil-Blind_Storage engine, durable via its write-ahead log and encrypted at rest with Ascon-128.

4

Gemini Incident Summaries

Deltas crossing a threshold trigger the Gemini API, producing a structured, plain-language maintenance note for operations staff.

Sample alert: "Bridge Node #04 recorded a 34% rise in RMS vibration alongside a 5°C temperature drop over 15 minutes — possible fastener loosening under thermal stress."

REFERENCES

Research & Documentation

RESEARCH PAPERS

[1] Myers, E. W. “An O(ND) Difference Algorithm and Its Variations.”

Algorithmica, vol. 1, pp. 251–266, 1986. DOI: 10.1007/BF01840446.

Basis of the shortest-edit-script diff algorithm implemented by java-diff-utils and used for baseline drift detection.

[2] Ansah, M. R., Aboagye, I. A., Annan, M., Bamiebo, O. N., Sowah, N. L., Okae, P.

“LoRaWAN-Based IoT Monitoring Systems for Sustainable Smart City.” IEEE ICAST, 2024. DOI: 10.1109/ICAST61769.2024.10856482.

Supports the use of low-cost LoRaWAN sensor nodes reporting to a central server for public-infrastructure monitoring.

DOCUMENTATION

- java-diff-utils — official library docs & README: github.com/java-diff-utils/java-diff-utils
- Ascon lightweight cryptography — NIST Lightweight Cryptography project (ascon.iaik.tugraz.at)
- Yggdrasil-Blind_Storage — internal README.md and BUG_FIX_REPORT.md (project repository)
- Google Gen AI SDK for Java — official developer documentation (ai.google.dev)

